

ruledtext 1.2 — API Reference Manual

Synopsis

```
package require ruledtext 1.2
package require ruledtext::pdf 1.2    ;# optional, for PDF export
```

Both ensemble style and explicit style work:

```
ruledtext create .ed                ;# ensemble
ruledtext::create .ed              ;# explicit (also valid)
```

Widget Creation and Access

create

```
ruledtext create $path ?-option value ...?
```

Creates a ruled text widget. Returns `$path`.

The widget is a `ttk::frame` containing a `text` widget (`$path.txt`) and a `ttk::scrollbar` (`$path.sb`). The text widget gets horizontal line frames, a margin frame, and a named font.

Configuration is copied from `ruledtext::cfg` at creation time. Each instance is independent -- changing `cfg` afterwards does not affect existing widgets.

```
ruledtext create .ed
pack .ed -fill both -expand 1
```

Multiple instances with different settings:

```
ruledtext create .a                                ;# default: blue lines, margin
set ruledtext::cfg(showmargin) 0
set ruledtext::cfg(paperbg) "#f0f0ff"
ruledtext create .b                                ;# no margin, different bg
# No reset needed -- .a keeps its own copy
```

textwidget

```
ruledtext textwidget $path
```

Returns the path to the underlying Tk text widget (`$path.txt`). Use this for all standard text operations:

```
set txt [ruledtext textwidget .ed]
$txt insert end "Hello\n"
$txt get 1.0 end
$txt tag configure bold -font {Courier 12 bold}
$txt see end
$txt search -regexp {pattern} 1.0
```

The text widget is a real Tk `text` -- all Tk text commands work unchanged. `ruledtext` adds lines on top, it does not replace the widget.

Font

setFont

```
ruledtext setFont $path $fontspec
```

Changes the base font. Uses a named font internally, so the line grid updates automatically. \$fontspec is any valid Tk font specification:

```
ruledtext setFont .ed {Courier 14}  
ruledtext setFont .ed {Helvetica 12 bold}  
ruledtext setFont .ed TkFixedFont
```

The named font is RuledFont_\$path. All tags using this font update simultaneously. The horizontal line spacing recalculates from the new font metrics -linespace.

Limitation: Tags with a different -font option will not align with the line grid.

Horizontal Lines

setLineColor

```
ruledtext setLineColor $path $color
```

Changes the color of all horizontal lines for this instance. Does not affect other instances (per-instance config).

```
ruledtext setLineColor .ed "#d0e8d0"    ;# green  
ruledtext setLineColor .ed "#e0e0e0"    ;# light grey  
ruledtext setLineColor .ed red           ;# named color
```

setLineRange

```
ruledtext setLineRange $path $from $to
```

Restricts horizontal lines to a pixel range (left to right). \$from = start x, \$to = end x. Value 0 as end means right widget edge.

```
ruledtext setLineRange .ed 0 0           ;# full width (default)  
ruledtext setLineRange .ed 55 0          ;# start at margin position  
ruledtext setLineRange .ed 20 0          ;# start 20px from left  
ruledtext setLineRange .ed 200 500       ;# only between 200-500px
```

setLinePattern

```
ruledtext setLinePattern $path $patternList
```

Sets a cyclic color pattern for horizontal lines. Each element is a color for one line. An empty string "" makes that line invisible (gap). Empty list {} restores uniform color from setLineColor.

```
ruledtext setLinePattern .ed {}                ;# uniform (default)
ruledtext setLinePattern .ed {"#d0d8e8" "#b0c0d8"} ;# alternating
ruledtext setLinePattern .ed {"#e0e0e0" "#e0e0e0" "#e0e0e0" "#e0e0e0" "#a0a0d0"}
                                ;# every 5th darker
ruledtext setLinePattern .ed {"#b0b0b0" "#b0b0b0" "#b0b0b0" "#b0b0b0" "#b0b0b0" "" "" ""}
                                ;# music staff (5 on, 3 off)
```

Presets can define `linepattern` as a dict key. The `music` preset uses a 5-on-3-off staff pattern by default.

setGridSize

```
ruledtext setGridSize $path $mode
```

Controls horizontal line spacing.

Mode	Spacing	Use case
"" (empty)	font metrics -linespace	Normal ruled paper (default)
"char"	font measure "M" (em width)	Experimental character grid

For square grids, use the `squared` or `graph` preset instead. They use `linespace` for both axes (truly square cells).

Left Margin

toggleMargin

```
ruledtext toggleMargin $path $bool
```

Shows or hides the vertical margin line (the red line on notebook paper). Also adjusts `padx` of the text widget: margin on = `marginx + 12`, margin off = `12`. Note: Tk's `text -padx` is symmetric (same value left and right), so the margin affects both sides equally.

```
ruledtext toggleMargin .ed 1 ;# show margin
ruledtext toggleMargin .ed 0 ;# hide margin, text shifts left
```

The margin position is set by `cfg(marginx)` before `create` (default: 55 pixels from left edge). To change it after create, use `setMarginX`.

setMarginX

```
ruledtext setMarginX $path $pixels
```

Moves the margin line to a new pixel position. Updates the internal state, text widget `padx`, and margin line placement in one call.

```
ruledtext setMarginX .ed 40 ;# narrow margin
ruledtext setMarginX .ed 80 ;# wide margin
ruledtext setMarginX .ed 55 ;# back to default
```

setMarginColor

```
ruledtext setMarginColor $path $color
```

Changes the left margin line color. Accepts any Tk color value.

```
ruledtext setMarginColor .ed "#e88080"    ;# red (default)
ruledtext setMarginColor .ed "#8080e8"    ;# blue
ruledtext setMarginColor .ed "#80c080"    ;# green
```

Right Margin

toggleRMargin

```
ruledtext toggleRMargin $path $bool
```

Shows or hides the right margin line. The right margin is visual only (via `place`) and does not affect text padding. Tk's `text -padx` is symmetric (same value left and right), so only the left margin affects text inset.

Default is off (`showmargin 0`).

```
ruledtext toggleRMargin .ed 1      ;# show right margin
ruledtext toggleRMargin .ed 0      ;# hide right margin
```

setRMarginX

```
ruledtext setRMarginX $path $pixels
```

Sets the right margin position as distance from the right widget edge. Updates state and margin line placement. Does not affect text padding (right margin is visual only).

```
ruledtext setRMarginX .ed 20      ;# narrow: 20px from right edge
ruledtext setRMarginX .ed 40      ;# wider: 40px from right edge
ruledtext setRMarginX .ed 30      ;# back to default
```

setRMarginColor

```
ruledtext setRMarginColor $path $color
```

Changes the right margin line color. Independent of left margin color.

```
ruledtext setRMarginColor .ed "#e88080"    ;# red (same as left)
ruledtext setRMarginColor .ed "#8080e8"    ;# blue
```

Vertical Lines

addVLine

```
ruledtext addVLine $path $x ?$color?
```

Adds a vertical line at pixel position `$x`. Optional `$color` (default: `linecolor`). Unlike horizontal lines, vertical lines are created and destroyed dynamically.

```
ruledtext addVLine .ed 150          ;# default color
ruledtext addVLine .ed 300 "#cccccc" ;# explicit color
```

If `setTabSync` is on, `syncTabs` is called automatically after each `addVLine`.

clearVLines

```
ruledtext clearVLines $path
```

Removes all vertical lines from the widget. Does not affect the margin line or horizontal lines.

setVLineColor

```
ruledtext setVLineColor $path $color
```

Changes the color of all existing vertical lines at once.

```
ruledtext setVLineColor .ed "#d0d0d0"
```

Does not affect lines added later. Does not affect the margin line or horizontal lines.

setVLineRange

```
ruledtext setVLineRange $path $from $to
```

Restricts vertical lines and both margin lines to a pixel range (top to bottom). \$from = start y, \$to = end y. Value 0 as end means bottom widget edge.

```
ruledtext setVLineRange .ed 0 0      ;# full height (default)
ruledtext setVLineRange .ed 20 0     ;# start 20px from top
ruledtext setVLineRange .ed 100 300  ;# only between 100-300px
```

Applies to left margin, right margin, and all vertical lines equally.

Text and Content

clear

```
ruledtext clear $path
```

Deletes all text content (\$path.txt delete 1.0 end). Lines, margin, and configuration remain unchanged.

insertText

```
ruledtext insertText $path $index $text ?$tag?
```

Inserts text even when the widget is in readonly mode. Temporarily sets state to normal, inserts, then restores the previous state.

```
ruledtext setReadOnly .ed select
ruledtext insertText .ed end "New entry\n"      ;# works
ruledtext insertText .ed end "Tagged\n" important
```

setReadOnly

```
ruledtext setReadOnly $path $mode
```

Mode	Editing	Selection	Use case
0	Yes	Yes	Normal editing (default)
1	No	No	Full readonly (disabled)

select	No	Yes (mouse)	Viewer, log display, help
--------	----	-------------	---------------------------

Mode `select` uses `bindtags` to block keyboard input while allowing mouse selection and copy (Ctrl+C).

```
ruledtext setReadOnly .ed 1      ;# disabled
ruledtext setReadOnly .ed select ;# read-only but selectable
ruledtext setReadOnly .ed 0      ;# back to normal
```

Tab Synchronization

syncTabs

```
ruledtext syncTabs $path
```

One-time synchronization: calculates tab stop positions from the current vertical line positions and applies them to the text widget.

The formula:

```
tabpos = vline_x - borderwidth + offset
```

With multiple vlines, `offset = (colwidth / 2) + 2` centers text in each column. With a single vline, `offset` uses `cfg(taboffset) + 2`.

```
ruledtext addVLine .ed 150
ruledtext addVLine .ed 320
ruledtext addVLine .ed 460
ruledtext syncTabs .ed      ;# text centered in columns
```

Sets `-tabstyle wordprocessor` automatically (absolute tab positions, like Word -- not relative to previous text).

setTabSync

```
ruledtext setTabSync $path $bool
```

Enables or disables automatic tab sync. When on, every `addVLine` call triggers `syncTabs` automatically.

```
ruledtext setTabSync .ed 1      ;# auto-sync on
ruledtext addVLine .ed 200      ;# -> tabs update
ruledtext addVLine .ed 400      ;# -> tabs update again

ruledtext setTabSync .ed 0      ;# auto-sync off
```

Presets

presetNames

```
ruledtext presetNames
```

Returns a sorted list of all available preset names:

```
set names [ruledtext presetNames]
# -> college columns dark graph green ledger minimal music squared todo vintage
```

preset

```
ruledtext preset $path $name
```

Applies a named preset. Resets everything: clears vertical lines, resets grid mode, resets line range/pattern, resets right margin (off), changes font, colors, margin.

```
ruledtext preset .ed college      ;# blue lines, red margin
ruledtext preset .ed dark         ;# dark background, muted lines
ruledtext preset .ed ledger       ;# columns with tab sync
```

Presets with `tabs 1` automatically enable tab sync and set `-tabstyle wordprocessor`. After applying `ledger` or `columns`, Tab key jumps between columns.

The preset name is stored internally. For `squared` and `graph` presets, the vertical grid extends automatically on resize.

An unknown preset name raises an error with `errorCode {RULEDTEXT PRESET UNKNOWN}`; the message lists the available presets.

Built-in Presets (11)

Preset	Lines	Background	Margin	Font	Special
college	Blue #a8c0e0	Warm white	Red, on	Courier 12	--
vintage	Brown #c8b890	Aged #f5f0e0	Brown, off	Times 13	--
minimal	Light grey	White	Grey, off	Helvetica 11	--
dark	Dark #404050	Dark #2a2a35	Dark, on	Consolas 12	Light text
green	Green #a8d0a8	Light green	Green, on	Courier 11	--
squared	Blue-grey	Blue-white	Off	Courier 12	Square grid
graph	Green-grey	Green-white	Off	Courier 10	Fine square grid
columns	Grey	White	Off	Helvetica 11	3 columns, tabs
ledger	Grey	Warm white	Red, on	Courier 11	4 columns, tabs
music	Grey #b0b0b0	Warm yellow	Off	Courier 12	Staff pattern (5 on, 3 off)
todo	Green-grey	Off-white	Off	Helvetica 11	Checkbox column

Custom presets can define the following dict keys: `linecolor`, `paperbg`, `margincolor`, `showmargin`, `font`, `fg`, `vgrid`, `vcols`, `vcolor`, `tabs`, `gridsize`, `linefrom`, `lineto`, `vlinefrom`, `vlineto`, `linepattern`, `showrmargin`, `rmarginx`, `rmargincolor`.

PDF Export

exportPDF

```
ruledtext exportPDF $path $filename ?-paper a4? ?-margin {50 50 50 50}? ?-title ""
```

Exports the widget to a PDF file. Returns the number of pages written.

All visual settings are reproduced: horizontal lines (color, range, pattern), left and right margin (position, color, on/off), vertical lines (positions, colors, range), background color, tabs, and text with automatic pagination.

```
# Load modules
tcl::tm::path add /path/to/ruledtext/lib
package require ruledtext 1.2
package require ruledtext::pdf 1.2

# Create widget and add content
ruledtext create .ed
set txt [ruledtext textwidget .ed]
$txt insert end "Content to export...\n"

# Export to PDF
ruledtext exportPDF .ed "output.pdf"
```

Options

Option	Default	Description
-paper	a4	Paper size (a4, letter)
-margin	{50 50 50 50}	Margins (left top right bottom, points)
-title	""	Header on each page

Page labels are auto-detected via `msgcat` (de: "Seite", en: "Page", fr: "Page", es: "Pagina", etc.).

Requirement: pdf4tcl 0.9+ (installed separately)

See `demos/demo-pdf-export.tcl` for a complete example.

Configuration Defaults

Set before `create` (copied per instance):

```
set ruledtext::cfg(linecolor)    "#c8d8e8"    ;# h-line color
set ruledtext::cfg(margincolor)  "#e8a0a0"    ;# left margin color
set ruledtext::cfg(marginx)      55            ;# left margin position (px)
set ruledtext::cfg(showmargin)   1            ;# left margin on/off
set ruledtext::cfg(rmargincolor) "#e8a0a0"    ;# right margin color
set ruledtext::cfg(rmarginx)     30            ;# right margin distance from right (p
set ruledtext::cfg(showrmargin)  0            ;# right margin on/off
set ruledtext::cfg(paperbg)      "#ffffff8"    ;# background
set ruledtext::cfg(maxlines)     80            ;# initial pool size
set ruledtext::cfg(taboffset)    0            ;# tab offset (px)
set ruledtext::cfg(linefrom)     0            ;# h-line start x
set ruledtext::cfg(lineto)       0            ;# h-line end x
set ruledtext::cfg(vlinefrom)    0            ;# v-line start y
set ruledtext::cfg(vlineto)      0            ;# v-line end y
set ruledtext::cfg(linepattern)  {}           ;# cyclic color list
```

Each instance gets its own copy at `create` time. After creation, use the API to change settings per

instance.

The global `cfg` is only a template for new instances.

Tab Position Formula

`place -x` measures from the widget edge (before `borderwidth`). `-tabs` measures from the text left edge (after `padx` and `borderwidth`). Both systems are already past `padx`, so only `borderwidth` matters:

```
tabpos = vline_x - borderwidth + offset
```

For text starting 1px right of the vertical line: `offset = 1`. For centered text in columns: `offset = (colwidth / 2) + 2`.

Important: Tab values must be separate list elements, not strings with `p` suffix:

```
# WRONG: "138p left" = points, not pixels; grows with distance
lappend tabs "${tabpos}p left"

# WRONG: "138 left" = one string element, not two list elements
lappend tabs "${tabpos} left"

# RIGHT: two separate elements, value in pixels
lappend tabs $tabpos left
```

Design Decisions

**Named Fonts:* Each instance creates a named font (`RuledFont_$path`). `setFont` uses `font configure` so all widgets using that font update automatically. Deleted on `<Destroy>`.

**\$txt sync before dlineinfo:* Ensures layout calculations are complete before querying pixel positions. Prevents incorrect line placement in large documents.

**-insertunfocussed hollow:* Cursor stays visible as hollow rectangle when the widget loses focus. Essential for split-view (demo-split).

**-tabstyle wordprocessor:* Set automatically for presets with `tabs 1`. Better column behavior than the default `tabular` style.

**<Destroy> Cleanup:* `SB` removes all ``state($path,)`` entries and the named font when the widget is destroyed. Prevents memory leaks with multiple instances.

**source -encoding utf-8:* Always use `-encoding utf-8` with `source`. Required on Windows with Tcl 8.6 (system encoding = cp1252). Harmless on Tcl 9 where UTF-8 is the default.

**Per-Instance Configuration:* `SB` `cfg` is copied into ``state($path, cfg, )oncreate``. Setters modify only the target instance. No cross-widget leaks.

**Dynamic Pool Growth:* The horizontal line pool starts at `maxlines` (default 80) but grows automatically in `_draw` when more lines are needed. Handles large windows, small fonts, and HiDPI without user config.

**Resize-Aware vgrid Presets:* `squared` and `graph` presets store their name in `lastpreset`. On resize, `_draw` calls `_reapplyVGrid` to extend the grid.

**Procedural Design:* No TclOO. Zero dependencies beyond Tk, low barrier to entry, simple `source`

-encoding utf-8 workflow.

padx limitation. Tk's `text -padx` accepts only a single value (symmetric left/right). The left margin affects text padding via `-padx`, but the right margin is visual only (placed via `:place`) and does not affect text inset.

Key Problems and Solutions

Problem	Solution
place frames cover text	Factor 1.5 positions lines between rows
Proc name shadows Tk command	Use <code>::place</code> globally qualified
Squares too narrow	Use linespace for both axes
Tab offset from vlines	Only subtract borderwidth, not padx
Tab values as points	No p suffix; bare number = pixels
Tab list element quoting	<code>lappend tabs \$val left</code> not <code>"\$val left"</code>
Memory leak on destroy	<code><Destroy></code> binding cleans up state + font
Layout race in <code>dlineinfo</code>	<code>\$txt sync</code> before position queries
cp1252 corruption on Windows	source -encoding utf-8 everywhere
Config leaks between instances	Per-instance cfg copy in state
Pool too small for large windows	Dynamic pool growth in <code>_draw</code>
vgrid breaks on resize	<code>_reapplyVGrid</code> extends grid automatically
Draw after widget destroyed	<code>info exists</code> + <code>wininfo exists</code> guards

Quick Reference

Proc	Args	Description
<code>create</code>	<code>path ?opts?</code>	Create widget
<code>textwidget</code>	<code>path</code>	Get text widget path
<code>setFont</code>	<code>path fontspec</code>	Change font
<code>setLineColor</code>	<code>path color</code>	H-line color
<code>setLineRange</code>	<code>path from to</code>	H-line x-range
<code>setLinePattern</code>	<code>path list</code>	H-line color cycle
<code>toggleMargin</code>	<code>path bool</code>	Left margin on/off
<code>setMarginX</code>	<code>path px</code>	Left margin position
<code>setMarginColor</code>	<code>path color</code>	Left margin color
<code>toggleRMargin</code>	<code>path bool</code>	Right margin on/off
<code>setRMarginX</code>	<code>path px</code>	Right margin distance from right
<code>setRMarginColor</code>	<code>path color</code>	Right margin color

clear	path	Delete all text
setReadOnly	path mode	Readonly (0/1/select)
insertText	path idx text ?tag?	Insert text
setGridSize	path mode	Grid spacing
addVLine	path x ?color?	Add vertical line
clearVLines	path	Remove all vlines
setVLineColor	path color	All vlines color
setVLineRange	path from to	V-line y-range
syncTabs	path	Sync tabs to vlines
setTabSync	path bool	Auto tab sync
presetNames	--	List preset names
preset	path name	Apply preset
exportPDF	path file ?opts?	Export to PDF

25 public procs, 7 internal procs (_sched, _draw, _growPool, _onScroll, _reapplyVGrid, _cleanup, _updatePdx).

Requirements

- Tcl/Tk 8.6.9+ or 9.0+
- pdf4tcl 0.9+ (only for PDF export)

Version

ruledtext 1.2